

CFRM 421/521, Spring 2025

[Eunice Seung-hyun Han]

Homework 2

- **Due: Tuesday, April 29, 2024, 11:59 PM**
- Total marks: 45
- Late submissions are allowed, but a 20% penalty per day applies. Your last submission is considered for calculating the penalty.
- Use this Jupyter notebook as a template for your solutions. **Your solution must be submitted as both one Jupyter notebook and one PDF file on Gradescope.** There will be two modules on Gradescope, one for each file type. The notebook must be already run, that is, make sure that you have run all the code, save the notebook, and then when you reopen the notebook, checked that all output appears as expected. You are allowed to use code from the textbook, textbook website, or lecture notes.

1. Random forest for time series data [14 marks]

In this question you will work with the NYSE dataset. Only 3 time series in this dataset will be use: `DJ_return` (a_t), `log_volatility` (b_t), and `log_volume` (c_t). Download the data as a csv file from [Canvas](#). The data was originally obtained from the R library ISLR2, and you can read the documentation for the dataset [here](#), which explains the meaning of the variables.

You want to predict the 1-step ahead value of `log_volume` c_{t+1} using the previous values of this variable and the other two variables (`DJ_return` and `log_volatility`) up to 5 lags. So the features are $c_t, \dots, c_{t-4}, b_t, \dots, b_{t-4}, a_t, \dots, a_{t-4}$.

If the data is stored in a file named `NYSE.csv` in your working directory, then loading the data can be done using the code below.

```
In [ ]: import os
import pandas as pd
data = pd.read_csv("NYSE.csv")
```

(a) [3 marks]

Create the feature matrix `X` and the target variable `y`. Print at least the first 2 rows of `X` and `y` (it is acceptable that not every element of the rows are printed).

Solution:

```
In [ ]: data = data[['log_volume', 'log_volatility', 'DJ_return']]
```

```
In [ ]: import numpy as np
def feature_target(data, feature_steps):
    DJ_return = data['DJ_return'].values
    log_volume = data['log_volume'].values
    log_volatility = data['log_volatility'].values

    X_l = []
    for t in range(feature_steps, len(log_volume)):
        row = []
        for var in [log_volume, log_volatility, DJ_return]:
            data1 = var[(t-feature_steps) : t]
            data1 = reversed(data1)
            row.extend(data1)
        X_l.append(row)
    X_array = np.array(X_l)
    y = log_volume[feature_steps:]

    return X_array, y
```

```
In [ ]: X, y = feature_target(data, 5)

print("First two rows of X: ")
print(X[:2])
print("")
print("First two rows of y: ")
print(y[:2])
```

First two rows of X:

```
[ [ 4.41870000e-02  2.10182000e-01  5.25306000e-01  3.46202000e-01
    3.25730000e-02 -1.17281302e+01 -1.16267724e+01 -1.16656090e+01
    -1.17493047e+01 -1.31274026e+01  5.68000000e-04 -3.46200000e-03
    3.84500000e-03  7.81300000e-03 -4.46100000e-03]
  [ 1.33246000e-01  4.41870000e-02  2.10182000e-01  5.25306000e-01
    3.46202000e-01 -1.08725263e+01 -1.17281302e+01 -1.16267724e+01
    -1.16656090e+01 -1.17493047e+01 -1.08240000e-02  5.68000000e-04
    -3.46200000e-03  3.84500000e-03  7.81300000e-03]]
```

First two rows of y:

```
[ 0.133246 -0.011528]
```

(b) [5 marks]

Consider fitting a random forest to predict the 1-step ahead value of `log_volume`. The random forest must include the argument `random_state=42`, and it is useful to also include `n_jobs=-1` (you can use `n_job=-1` throughout this homework wherever it is available). Use 3-fold time series CV split, with the test set split 50% into a validation set and 50% into the actual test set, to tune the hyperparameters `n_estimators` taking the values 200, 400, 600, and the cost-complexity pruning parameter α taking the values 10^{-k} , $k = 1, 3, 5, 7$. When tuning hyperparameters on the validation sets, fit the model only on a random 10% sample of the instances of the training set on the same CV fold to reduce computational time (that is, use the same reduced training set for all the hyperparameters, but a different one for each CV fold). Note this will still preserve the correct time ordering, and the reduced training set should not be used when fitting and evaluating the best model on the test set. The performance measure is RMSE. Report the best hyperparameters.

Solution:

```
In [ ]: import numpy as np
        from sklearn.model_selection import TimeSeriesSplit, train_test_split
        from sklearn.ensemble import RandomForestRegressor
        from sklearn.metrics import mean_squared_error
        from itertools import product

        tscv = TimeSeriesSplit(n_splits=3)
        ccp_alpha_vals = [10*(-k) for k in (1,3,5,7)]
        n_estimators_vals = [200, 400, 600]
        param_grid = list(product(ccp_alpha_vals, n_estimators_vals))

        rmse_results = {params: [] for params in param_grid}

        for train_idx, test_idx in tscv.split(X):
```

```

# split test_idx into validation and test halves
half = len(test_idx) // 2
valid_idx = test_idx[:half]

X_train, y_train = X[train_idx], y[train_idx]
X_train_sub, _, y_train_sub, _ = train_test_split(X_train, y_train)
X_valid, y_valid = X[valid_idx], y[valid_idx]

for alpha, n_est in param_grid:
    rf = RandomForestRegressor(random_state=42, ccp_alpha=alpha, n_estimators=n_est)
    rf.fit(X_train_sub, y_train_sub)
    y_pred = rf.predict(X_valid)
    rmse = np.sqrt(mean_squared_error(y_valid, y_pred))
    rmse_results[(alpha, n_est)].append(rmse)

# 5) Compute average RMSE across folds
avg_rmse = {params: np.mean(scores) for params, scores in rmse_results.items()}

# 6) Select best
best_params = min(avg_rmse, key=avg_rmse.get)
print("Best hyperparameters (ccp_alpha, n_estimators):", best_params)
print("Validation RMSE:", avg_rmse[best_params])

```

Best hyperparameters (ccp_alpha, n_estimators): (1e-07, 600)
 Validation RMSE: 0.16938038878764058

(c) [2 marks]

Using the same time series split as in (b), compute the RMSE of the best fitting model on the test set, and include a plot of the true values and predicted values on the test set of the last fold (the fold closest to the current time) of the CV.

Solution:

```

In [ ]: import matplotlib.pyplot as plt

# grab the last fold..
splits = list(tscv.split(X))
train_idx, test_idx = splits[-1]

# split that fold's test indices 50/50..
half = len(test_idx) // 2
test_idx = test_idx[half:] # this is your final test set

X_train, y_train = X[train_idx], y[train_idx]
X_test, y_test = X[test_idx], y[test_idx]

best_alpha = best_params[0]; best_est = best_params[1]
model_best = RandomForestRegressor(random_state=42, ccp_alpha=best_alpha, n_estimators=best_est)

```

```

model_best.fit(X_train, y_train)
y_pred = model_best.predict(X_test)
rmse_test = np.sqrt(mean_squared_error(y_test, y_pred))
print("RMSE of the best fitting model on the Test set: ", rmse_test)

plt.figure()
plt.plot(y_test, label="True", color='lightblue')
plt.plot(y_pred, label="Predicted", color='orange')
plt.xlabel("Test sample index")
plt.ylabel("log_volume")
plt.title('True vs Predicted values on the Test set')
plt.legend()
plt.show()

```

RMSE of the best fitting model on the Test set: 0.21586423341636898



(d) [2 marks]

It is often useful to check that your model is not worse than a very simple method of prediction. On the test set, compute the RMSE of a model that simply predicts the 1-step ahead value of `log_volume` c_{t+1} as the current value c_t , and compare this to the best fitting random forest model.

Solution:

```
In [ ]: from sklearn.metrics import mean_squared_error

baseline_rmse = []

for train_idx, test_idx in tscv.split(X):
    half = len(test_idx) // 2
    test_idx2 = test_idx[half:]

    y_true = y[test_idx2]
    y_naive = X[test_idx2][:, 0]

    # Compute RMSE for this fold
    rmse_naive = np.sqrt(mean_squared_error(y_true, y_naive))

# Results
print("Best fitting Random Forest model RMSE:", rmse_test)
print("Naïve Baseline model RMSE:", rmse_naive)
```

Best fitting Random Forest model RMSE: 0.21586423341636898
 Naïve Baseline model RMSE: 0.2534597191392435

The RMSE of the best fitting random forest model is smaller than the naive baseline model (that simply predicts the 1-step ahead value of c_{t+1} as the current value c_t). This tells the best fitting random forest model has a higher prediction accuracy than the naive baseline model.

(e) [2 marks]

Compute the feature importances of the best fitting model. Which feature is the most important and what is its feature importance value?

Solution:

```
In [ ]: best_ne, best_alpha = best_params[1], best_params[0]
best_rf_model = RandomForestRegressor(ccp_alpha=best_alpha, n_estimators=best_ne)

for train_index, test_index in tscv.split(X):
    X_test1 = X[test_index]; y_test1 = y[test_index]
    half = len(test_index) // 2
    X_test = X_test1[half:]; y_test = y_test1[half:]
best_rf_model.fit(X_test, y_test)
importances = best_rf_model.feature_importances_
most_important = np.argmax(importances)
print("Best parameters (ccp_alpha, n_estimators) : ", best_params)
print("The index that has the most important feature: ", most_important)
print("The most important feature's value: ", importances[most_important])

vars_ = ['log_volume', 'log_volatility', 'DJ_return']
feature_names = [
```

```

    f"{var}_t{lag}"
    for lag in range(5)
    for var in vars_
]

# sanity check
assert len(importances) == len(feature_names) == 15

imp_series = pd.Series(importances, index=feature_names)
imp_series = imp_series.sort_values(ascending=False)

# --- 3) Report the top feature ---
top_feat = imp_series.index[0]
top_value = imp_series.iloc[0]

print("The feature that is the most important:", f" {top_feat} → {top

```

```

Best parameters (ccp_alpha, n_estimators) : (1e-07, 600)
The index that has the most important feature: 0
The most important feature's value: 0.36024101145405124
The feature that is the most important: log_volume_t0 → 0.3602

```

Through the results, I could observe that the c_t , which the log_volume data right before predicting $c_{(t+1)}$, is the most important feature.

2. SVM classification and regression [11 marks]

For all SVM models in this question use a standard scaler.

(a) [2 marks]

In this question, a SVM is used for classification for the MNIST dataset. The following code loads the MNIST dataset, creates the test set, and to reduce training time, takes a random sample of 2000 points from the full training set to use as your actual training set stored in `X` and `y`. Do not shuffle the data.

Hint: Reading the solution to Question 9 in the Chapter 5 [Jupyter notebook](#) (2nd edition) on the textbook website may help with this question.

```

In [ ]: import numpy as np
import matplotlib as mlp
import pandas as pd

from sklearn.datasets import fetch_openml

```

```
mnist = fetch_openml('mnist_784', as_frame=False, cache=True, parser='
X_train = mnist["data"][:60000]
X_test = mnist["data"][60000:]
y_train = mnist["target"][:60000]
y_test = mnist["target"][60000:]
```

```
In [ ]: from sklearn.model_selection import StratifiedShuffleSplit

N = 2000
split_obj = StratifiedShuffleSplit(n_splits=1,
                                   test_size=N/60000, random_state=42)
for other_idx, subsample_idx in split_obj.split(X_train, y_train):
    X = X_train[subsample_idx]
    y = y_train[subsample_idx]
```

Task: Consider fitting the linear SVM classifier (`LinearSVC`) with `max_iter=50000` . For this model, optimize the hyperparameter C using 3-fold CV over the values 10^{-k} , $k = 0, 1, \dots, 9$, where the performance measure is accuracy. What is the best C and what is the accuracy in this case?

Solution:

```
In [ ]: from sklearn.preprocessing import StandardScaler
from sklearn.svm import LinearSVC
from sklearn.model_selection import GridSearchCV
from sklearn.model_selection import RandomizedSearchCV

scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X)
X_test_scaled = scaler.fit_transform(X_test)

svc = LinearSVC(max_iter=50000, random_state=42, dual=True)

param_grid = {'C': [10*(-k) for k in range(10)]}
grid = RandomizedSearchCV(estimator=svc, param_distributions=param_gri
grid.fit(X_train_scaled, y)

best_C = grid.best_params_['C']
best_acc = grid.best_score_

print(f"Best hyperparameter C: {best_C}")
print(f"The accuracy with best hyperparameter: {best_acc}")
```

Best hyperparameter C: 0.01

The accuracy with best hyperparameter: 0.8284896590743668

(b) [2 marks]

Task: Now consider fitting a SVM with a Gaussian RBF kernel and `max_iter=50000`. For this model, optimize the hyperparameters C over the distribution `uniform(1,10)` and γ over the distribution `loguniform(0.0001, 0.1)` from `scipy.stats.loguniform` with 10 random samples. The `loguniform(a,b)` function takes a random sample from the probability distribution with pdf $f(x) \propto 1/x, x \in [a, b]$. Again, use 3-fold CV and the performance measure is accuracy. What are the best hyperparameters and what is the accuracy in this case?

Solution:

```
In [ ]: from sklearn.svm import SVC
        from scipy.stats import uniform, loguniform

        svc = SVC(kernel='rbf', max_iter=50000)
        param_distributions = {'C': uniform(1, 10), 'gamma': loguniform(0.0001, 0.1)}

        random_search = RandomizedSearchCV(estimator=svc, param_distributions=param_distributions,
                                            cv=3, scoring='accuracy', random_state=42)

        random_search.fit(X_train_scaled, y)

        random_best_params = random_search.best_params_
        random_best_score = random_search.best_score_

        random_best_params = {k: float(v) for k, v in random_best_params.items()}

        print("Best hyperparameters: ", random_best_params)
        print(f"The accuracy with best hyperparameters: {random_best_score}")
```

Best hyperparameters: {'C': 9.324426408004218, 'gamma': 0.0004335281794951569}

The accuracy with best hyperparameters: 0.8964999482240862

(c) [2 mark]

Task: Choose the best model in (a) and (b). Then for this model, evaluate the accuracy on the test set, which is stored in `X_test` and `y_test`.

Solution:

```
In [ ]: best_model = random_search.best_estimator_
        best_model
```

Out []:

```
SVC
```

```
SVC(C=np.float64(9.324426408004218), gamma=np.float64(0.0004335281794951569),
    max_iter=50000)
```

In []:

```
from sklearn.metrics import accuracy_score

y_test_prediction = best_model.predict(X_test_scaled)
accuracy = accuracy_score(y_test, y_test_prediction)
print("The accuracy on the test set of the best model: ", accuracy)
```

The accuracy on the test set of the best model: 0.9163

(d) [3 marks]

Consider the original source of the California housing data (which is different from the modified dataset used in Homework 1) in Scikit-Learn. The data is obtained and split using the code below. The training set is stored in `X_train` and `y_train`. Do not shuffle the data.

Hint: Reading the solution to Question 11 in the Chapter 5 [Jupyter notebook](#) (3rd edition) on the textbook website may help with this question.

In []:

```
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split

housing = fetch_california_housing()
X = housing.data
y = housing.target

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.
```

Task: Consider SVM regression with a Gaussian RBF kernel and a sigmoid kernel with `max_iter=50000`. For both models, use randomized search to choose good hyperparameter values for `C` and `gamma`, and set the argument `random_state=42`. For both models, optimize the hyperparameters C over the distribution `uniform(1,20)` and γ over the distribution `loguniform(0.0001, 0.1)` with 20 random samples. To save training time, use only the first 2000 instances of `X_train` and `y_train` (which have been randomly shuffled already) for the search. Again, use 3-fold CV and the performance measure is MSE. What are the best hyperparameters and what is the MSE in this case?

Solution:

```
In [ ]: scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)
X_train_sub = X_train_scaled[:2000]
y_train_sub = y_train[:2000]
print(X_train_sub)

[[-0.326196    0.34849025 -0.17491646 ...  0.05137609 -1.3728112
  1.27258656]
 [-0.03584338  1.61811813 -0.40283542 ... -0.11736222 -0.87669601
  0.70916212]
 [ 0.14470145 -1.95271028  0.08821601 ... -0.03227969 -0.46014647
 -0.44760309]
 ...
 [ 0.32955246  1.06265593  0.08577253 ... -0.06451337 -1.32132755
  1.1529212 ]
 [-0.76012325  0.74524896 -0.18941217 ... -0.02667258  0.99075643
 -1.29024672]
 [-0.70545567 -1.39724809  0.16463454 ... -0.07636597  1.94086381
 -1.01601359]]
```

```
In [ ]: from sklearn.svm import SVR

param_distributions = {'C': uniform(1,20), 'gamma': loguniform(0.0001,
SVR_rbf = SVR(kernel='rbf', max_iter=50000)
random_SVR_rbf = RandomizedSearchCV(SVR_rbf, param_distributions=param
                                   cv=3, scoring='neg_mean_squared_er

random_SVR_rbf.fit(X_train_sub, y_train_sub)

random_SVR_rbf_MSE = -random_SVR_rbf.best_score_

# SVM regression with a Gaussian RBF kernel
print('Best hyperparameters of SVM regression with a Gaussian RBF kern
print('MSE of SVM regression with a Gaussian RBF kernel with best hype
```

```
Best hyperparameters of SVM regression with a Gaussian RBF kernel: {'
C': np.float64(8.49080237694725), 'gamma': np.float64(0.071144760093434
25)}
```

```
MSE of SVM regression with a Gaussian RBF kernel with best hyperparamet
ers:  0.35499207529804505
```

```
In [ ]: SVR_sig = SVR(kernel='sigmoid', max_iter=50000)
random_SVR_sig = RandomizedSearchCV(SVR_sig, param_distributions=param
                                   cv=3, scoring='neg_mean_squared_er

random_SVR_sig.fit(X_train_sub, y_train_sub)

random_SVR_sig_MSE = -random_SVR_sig.best_score_

# SVM regression with a Sigmoid kernel
print('Best hyperparameters of SVM regression with a Sigmoid kernel: '
print('MSE of SVM regression with a Sigmoid kernel with best hyperpara
```

Best hyperparameters of SVM regression with a Sigmoid kernel: {'C': np.float64(14.684660530243137), 'gamma': np.float64(0.0020914981329035616)}

MSE of SVM regression with a Sigmoid kernel with best hyperparameters: 0.4924854999249508

(e) [2 marks]

Task: Choose the best model in (d). But now refit it on the full training set (not just the first 2000 instances). Then for this model, evaluate the RMSE on the test set, which is stored in `X_test` and `y_test`.

Solution:

```
In [ ]: best_model = random_SVR_rbf.best_estimator_
        best_model
```

```
Out[ ]: SVR
SVR(C=np.float64(8.49080237694725), gamma=np.float64(0.07114476009343425),
    max_iter=50000)
```

```
In [ ]: best_model.fit(X_train_scaled, y_train)
        y_test_prediction = best_model.predict(X_test_scaled)
```

```
/Users/eunicehan/opt/anaconda3/envs/py312/lib/python3.12/site-packages/
sklearn/svm/_base.py:305: ConvergenceWarning: Solver terminated early (
max_iter=50000). Consider pre-processing your data with StandardScaler
or MinMaxScaler.
    warnings.warn(
```

```
In [ ]: RMSE = np.sqrt(mean_squared_error(y_test, y_test_prediction))
        print("RMSE on the test set: ", RMSE)
```

RMSE on the test set: 0.5919721441513734

3. Voting classifiers [11 marks]

(a) [4 marks]

Consider the MNIST dataset. To save computational time, after splitting into a training, validation and test set, we keep only the first 5000 instances of the training set, and only the first 1000 instances of the validation and test set, as given by the following code.

```
In [ ]: N = 50_000
M = 60_000
X_train = mnist["data"][:N][:5000]
y_train = mnist["target"][:N][:5000]
X_valid = mnist["data"][N:M][:1000]
y_valid = mnist["target"][N:M][:1000]
X_test = mnist["data"][M:][:1000]
y_test = mnist["target"][M:][:1000]
```

Do not shuffle the data and do not use a standard scaler. Train the following classifiers on the training set:

(i) a multilayer perceptron classifier using the class `MLPClassifier()` from `sklearn.neural_network` with arguments `random_state=42`,

(ii) an extra-trees classifier with arguments `n_estimators=100`, `n_jobs=-1`, `random_state=42`,

(iii) an AdaBoost classifier with arguments `n_estimators=50`, `learning_rate=0.2`, `random_state=42`,

(iv) a gradient boosting classifier using the class `GradientBoostingClassifier()` with arguments `max_depth=2`, `n_estimators=10`, `learning_rate=0.25`, `random_state=42`.

Report the accuracy of each trained classifier on the validation set.

Hint: Reading the solution to Question 8 in the Chapter 7 [Jupyter notebook](#) on the textbook website may help with this question.

Solution:

```
In [ ]: # (i)
from sklearn.neural_network import MLPClassifier
mlp_clf = MLPClassifier(random_state=42)
mlp_clf.fit(X_train, y_train)
accuracy_mlp = mlp_clf.score(X_valid, y_valid)

# (ii)
from sklearn.ensemble import ExtraTreesClassifier
extra_trees_clf = ExtraTreesClassifier(n_estimators=100, random_state=42)
extra_trees_clf.fit(X_train, y_train)
accuracy_extra_trees = extra_trees_clf.score(X_valid, y_valid)

# (iii)
from sklearn.ensemble import AdaBoostClassifier
AdaB_clf = AdaBoostClassifier(n_estimators=50, learning_rate=0.2, random_state=42)
AdaB_clf.fit(X_train, y_train)
```

```

accuracy_AdaB = AdaB_clf.score(X_valid, y_valid)

# (iv)
from sklearn.ensemble import GradientBoostingClassifier
GB_clf = GradientBoostingClassifier(max_depth=2, n_estimators=10, learning_rate=0.1)
GB_clf.fit(X_train, y_train)
accuracy_GB = GB_clf.score(X_valid, y_valid)

print("Accuracy of validation set when using ... ")
print("Multilayer Perceptron Classifier: ", accuracy_mlp)
print("Extra Trees Classifier: ", accuracy_extra_trees)
print("AdaBoost Classifier: ", accuracy_AdaB)
print("Gradient Boosting Classifier: ", accuracy_GB)

```

Accuracy of validation set when using ...
 Multilayer Perceptron Classifier: 0.878
 Extra Trees Classifier: 0.944
 AdaBoost Classifier: 0.431
 Gradient Boosting Classifier: 0.802

(b) [5 marks]

Train the following models:

- a hard-voting ensemble classifier for all the models in (a)
- a soft-voting ensemble classifier for all the models in (a)
- a hard-voting ensemble classifier dropping the worst performing model in (a)
- a soft-voting ensemble classifier dropping the worst performing model in (a)

Evaluate the accuracy of these voting classifiers on the validation set, and compare it to the performance of the individual models in (a).

Solution:

```

In [ ]: from sklearn.ensemble import VotingClassifier

# All classifiers
estimators_all = [('Multilayer Perceptron Classifier', mlp_clf), ('Extra Trees Classifier', extra_trees_clf), ('AdaBoost Classifier', AdaB_clf), ('Gradient Boosting Classifier', GB_clf)]

# (1) Hard-voting with all the models
voting_clf_hard_all = VotingClassifier(estimators=estimators_all, voting='hard')
voting_clf_hard_all.fit(X_train, y_train)
accuracy_hard_all = voting_clf_hard_all.score(X_valid, y_valid)
print("Accuracy of a hard-voting ensemble classifier for all the models: ", accuracy_hard_all)

# (2) Soft-voting with all the models
voting_clf_soft_all = VotingClassifier(estimators=estimators_all, voting='soft')
voting_clf_soft_all.fit(X_train, y_train)
accuracy_soft_all = voting_clf_soft_all.score(X_valid, y_valid)
print("Accuracy of a soft-voting ensemble classifier for all the models: ", accuracy_soft_all)

```

```
accuracy_soft_all = voting_clf_soft_all.score(X_valid, y_valid)
print("Accuracy of a soft-voting ensemble classifier for all the model
```

Accuracy of a hard-voting ensemble classifier for all the models in (a) on the validation set 0.897

Accuracy of a soft-voting ensemble classifier for all the models in (a) on the validation set 0.886

```
In [ ]: accuracies = {'Multilayer Perceptron Classifier': accuracy_mlp, 'Extra
                  'AdaBoost Classifier': accuracy_AdaB, 'Gradient Boosting

worst_model = min(accuracies, key=accuracies.get)
print("Worst-performing model:", worst_model)

# Drop the worst
estimators_drop = [tup for tup in estimators_all if tup[0] != worst_mo

# (3) Hard-voting without worst model
voting_clf_hard_drop = VotingClassifier(estimators=estimators_drop, vo
voting_clf_hard_drop.fit(X_train, y_train)
accuracy_hard_drop = voting_clf_hard_drop.score(X_valid, y_valid)
print("Accuracy of a hard-voting ensemble classifier dropping the wors

# (4) Soft-voting without worst model
voting_clf_soft_drop = VotingClassifier(estimators=estimators_drop, vo
voting_clf_soft_drop.fit(X_train, y_train)
accuracy_soft_drop = voting_clf_soft_drop.score(X_valid, y_valid)
print("Accuracy of a soft-voting ensemble classifier dropping the wors
```

Worst-performing model: AdaBoost Classifier

Accuracy of a hard-voting ensemble classifier dropping the worst performing model in (a) – AdaBoost Classifier , on the validation set: 0.915

Accuracy of a soft-voting ensemble classifier dropping the worst performing model in (a) – AdaBoost Classifier , on the validation set: 0.886

For the four following models,

- a hard-voting ensemble classifier for all the models in (a)
- a soft-voting ensemble classifier for all the models in (a)
- a hard-voting ensemble classifier dropping the worst performing model in (a)
- a soft-voting ensemble classifier dropping the worst performing model in (a)

I have found out that hard-voting ensemble classifier dropping the worst performing model in (a), which is the 'AdaBoost Classifier', got the highest accuracy score. Comparing the accuracy of the voting classifiers in (b) on the validation set with the performance of the individual models in (a); the accuracy of the Multilayer Perceptron, AdaBoost, and Gradient Boosting Classifier was lower in (a) than the accuracy of the voting classifiers in (b) but except the Extra Trees Classifier. The Extra Trees Classifier had a particularly higher accuracy score in (a) compared to the voting classifiers in (b) and also among all of the

models.

(c) [2 marks]

Of the four voting classifiers in (b), choose the best model. Then evaluate the accuracy of this model on the test set.

Solution:

```
In [ ]: voting_accuracies = {'Hard Voting (All)': accuracy_hard_all, 'Soft Voting (All)': accuracy_soft_all, 'Hard Voting (Drop)': accuracy_hard_drop, 'Soft Voting (Drop)': accuracy_soft_drop}

best_model_name = max(voting_accuracies, key=voting_accuracies.get)
print("Best Voting Model:", best_model_name)

if best_model_name == 'Hard Voting (All)':
    best_model = voting_clf_hard_all
elif best_model_name == 'Soft Voting (All)':
    best_model = voting_clf_soft_all
elif best_model_name == 'Hard Voting (Drop)':
    best_model = voting_clf_hard_drop
else:
    best_model = voting_clf_soft_drop

accuracy_test = best_model.score(X_test, y_test)
print(f"Test Accuracy of Best Voting Model, '{best_model_name}': {accuracy_test}")
```

Best Voting Model: Hard Voting (Drop)

Test Accuracy of Best Voting Model, 'Hard Voting (Drop)': 0.913

4. Stacking [9 marks]

We continue with the setting of Question 3. The training set, validation set and test set are the same. In Question 3, we have used predetermined rules (that is, hard-voting and soft-voting) to build the ensemble prediction. **Stacking** is an ensemble method in which you train a model (called a **blender**) to aggregate the result of each predictor into an ensemble prediction.

Hint: Reading the subsection "Stacking" in Chapter 7 of the textbook and the solution to Question 9 in the Chapter 7 [Jupyter notebook](#) on the textbook website may help with this question.

(a) [3 marks]

For each of the four classifiers in Question 3(a), make 5000 clean predictions on

the training set with 3-fold cross validation using

`sklearn.model_selection.cross_val_predict()`. You should end up with four predictions per observation. Print at least the first 5 rows of `pred`. Next, apply one-hot encoding to `pred` since these predictions are class labels.

Solution:

```
In [ ]: from sklearn.model_selection import cross_val_predict
        from sklearn.preprocessing import OneHotEncoder

        # Get predictions from each classifier
        pred_mlp = cross_val_predict(mlp_clf, X_train, y_train, cv=3)
        pred_et  = cross_val_predict(extra_trees_clf, X_train, y_train, cv=3)
        pred_ada = cross_val_predict(AdaB_clf, X_train, y_train, cv=3)
        pred_gb  = cross_val_predict(GB_clf, X_train, y_train, cv=3)

        # Stack predictions column-wise
        pred = np.array([pred_mlp, pred_et, pred_ada, pred_gb]).T

        # Print first 5 rows of raw predictions
        print("First 5 rows of raw predictions:")
        print(pred[:5])

        # One-hot encode the predictions
        encoder = OneHotEncoder()
        pred_encoded = encoder.fit_transform(pred).toarray()

        # Optionally, convert to DataFrame for easier inspection
        pred_encoded_df = pd.DataFrame(pred_encoded)
        print("\nFirst 5 rows of one-hot encoded predictions:")
        print(pred_encoded_df[:5])
```

First 5 rows of raw predictions:

```
[['5' '5' '0' '3']
 ['0' '0' '0' '0']
 ['4' '4' '7' '4']
 ['1' '1' '1' '1']
 ['9' '9' '4' '9']]
```

First 5 rows of one-hot encoded predictions:

	0	1	2	3	4	5	6	7	8	9	...	30	31	32
0	0.0	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
1	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	1.0	0.0	0.0
2	0.0	0.0	0.0	0.0	1.0	0.0	0.0	0.0	0.0	0.0	...	0.0	0.0	0.0
3	0.0	1.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	...	0.0	1.0	0.0
4	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	0.0	1.0	...	0.0	0.0	0.0

	34	35	36	37	38	39
0	0.0	0.0	0.0	0.0	0.0	0.0
1	0.0	0.0	0.0	0.0	0.0	0.0
2	1.0	0.0	0.0	0.0	0.0	0.0
3	0.0	0.0	0.0	0.0	0.0	0.0
4	0.0	0.0	0.0	0.0	0.0	1.0

[5 rows x 40 columns]

(b) [3 marks]

Use the predictions in (a) as features and the actual label of the observations as the target. Train a random forest classifier on the training set with the parameters `n_estimators=100`, `random_state=42`. This classifier is a blender.

Solution:

```
In [ ]: from sklearn.ensemble import RandomForestClassifier

# Step 1: Use the one-hot encoded predictions as features
X_blender = pred_encoded # from 4-(a)
y_blender = y_train      # actual training labels

# Step 2: Train the random forest blender
blender_clf = RandomForestClassifier(n_estimators=100, random_state=42)
blender_clf.fit(X_blender, y_blender)

train_accuracy = blender_clf.score(X_blender, y_blender)
print(f"Training accuracy of the blender: {train_accuracy}")
```

Training accuracy of the blender: 0.9624

(c) [3 marks]

Obtain the predictions of the blender on the test set by feeding predictions on the test set from the four classifiers in Question 3(a) into the blender trained in Question 4(b). Do not retrain the blender. These are called stacking predictions. Report the accuracy of your stacking predictions on the test set and compare this to the results in Question 3(c).

Solution:

```
In [ ]: from sklearn.metrics import accuracy_score

# Get predictions of base models on the test set
test_pred_mlp = mlp_clf.predict(X_test)
test_pred_et  = extra_trees_clf.predict(X_test)
test_pred_ada = AdaB_clf.predict(X_test)
test_pred_gb  = GB_clf.predict(X_test)

# Stack predictions
test_pred = np.column_stack((test_pred_mlp, test_pred_et, test_pred_ada, test_pred_gb))

# One-hot encode test predictions (use the encoder already fit on training set)
test_pred_encoded = encoder.transform(test_pred) # do not call fit_transform

# Get blender predictions
stacking_preds = blender_clf.predict(test_pred_encoded)

# Evaluate accuracy
stacking_accuracy = accuracy_score(y_test, stacking_preds)
print(f"Stacking prediction accuracy on Test set: {stacking_accuracy}")
```

Stacking prediction accuracy on Test set: 0.929

On the test set, the accuracy of stacking prediction is higher than the accuracy of the hard-voting ensemble classifier dropping the worst performing model (AdaBoost Classifier). Question 3(c) got the test accuracy 0.913 while in Question 4(c), the test accuracy is 0.929.